Phase 2 Report

Model Implementation, Pretraining and Evaluation

CL3410: Language Models and Agents, Monsoon 2026

Sourav Nath

Roll Number: 2025201054

31 August 2026

Models: Model H (Hindi, Devanagari) · Model L (Assamese, Bengali–Assamese script)

Summary

Two decoder-only Transformers of **24,086,016** trainable parameters each were implemented from primitive PyTorch layers and pretrained independently — Model H on Hindi, Model L on Assamese — on an *identical* budget of 589,824,000 tokens (6,000 optimiser steps × 98,304 tokens, about one epoch of each corpus). No component is shared between them: separate data, separate 16k tokenizers, separate vocabularies, separate weights, and neither was initialised from the other. On the held-out test split Model H reaches **0.5595** bits-per-byte and Model L **0.6238**. Because the token budget was matched almost exactly (Assamese has 0.2% more training tokens), the remaining gap is attributable to corpus *composition* and tokenizer fertility rather than to data quantity — see §6.

Pretrained checkpoints (Model H and Model L):
https://drive.google.com/drive/folders/1bIOG67ZM5iARPTvk3FGR51DxqIIxlLLn?usp=sharing

Contents

1 Architecture **2**
1.1 Configuration . . . 2
1.2 Depth/width trade-off . . . 3
1.3 Forward pass . . . 3
1.4 Empirical verification . . . 4

2 Pretraining **5**
2.1 Setup . . . 5
2.2 Checkpoints . . . 6
2.3 Loss curves . . . 6

3 Intrinsic evaluation **7**
3.1 Why bits-per-byte, and how to read the gap . . . 8

4 Generation quality **8**
4.1 What these metrics do and do not tell us 8
4.2 Diversity and repetition 9
4.3 Qualitative samples 9

5 Attention analysis **10**
5.1 What the heads learned 12
5.2 Where the two models differ 13

6 Resource-level comparison **14**

7 Reproduction and artifacts **15**
7.1 In this repository (branch `phase-2`) 15
7.2 On Google Drive 15
7.3 Reproducing 15

1 Architecture

The model is a GPT-style decoder-only Transformer written from `nn.Linear`, `nn.Embedding`, `nn.LayerNorm` and `nn.Dropout` only. No `nn.Transformer*` module, no HuggingFace model class and no pre-built attention block appears anywhere in `common/modeling/model.py`.

1.1 Configuration

Both languages use the same architecture; only the weights differ. The vocabulary is 16,000 for both, so the parameter count is identical.

Table 1: Architecture, identical for Model H and Model L.

Hyperparameter	Value
Layers N	10
Attention heads h	6
Model dimension d_{model}	384
Head dimension $d_k = d_{\text{model}}/h$	64
Feed-forward width d_{ff}	1536 ($4 \times d_{\text{model}}$)
Context length	512 tokens
Vocabulary	16,000
Positional encoding	learned absolute
Normalisation	pre-norm, plus a final LayerNorm
Activation	GELU
Dropout	0.1 (embeddings, attention probabilities, sublayer outputs)
Input/output embeddings	**tied**
Total trainable parameters	**24,086,016**
token embedding (tied, counted once)	6,144,000 (25.5%)
positional embedding	196,608 (0.8%)
Transformer blocks	17,744,640 (73.7%)
final LayerNorm	768

1.2 Depth/width trade-off

At a fixed ∼25M budget, depth and width trade against each other: block parameters scale as $N \cdot (4d^2 + 2d\,d_{\text{ff}}) = 12Nd^2$ when $d_{\text{ff}} = 4d$. Ten layers at $d = 384$ was chosen over a shallower, wider alternative (say 6 layers at $d = 512$, which lands at a similar count) because depth buys *composition*: each block can condition on the representation the previous one built, which is what lets later layers form the long-range, content-based heads observed in §5. Width mainly buys capacity per position. For a model this small trained on a single epoch, the binding constraint is representational structure rather than raw capacity, so depth was favoured. Six heads at $d_k = 64$ keeps each head's subspace at the size where scaled dot-product attention behaves well; going to 12 heads at $d_k = 32$ would have made each head's similarity computation noticeably noisier.

1.3 Forward pass

Input representation. Token ids index an embedding matrix of shape $(16000, 384)$. Learned absolute positional embeddings of shape $(512, 384)$ are added, then dropout. Learned absolute encodings were chosen over sinusoidal for simplicity and over RoPE to keep the implementation transparent; the cost is that the model *cannot* extrapolate beyond 512 positions, because position 512 has no learned vector. The implementation raises a `ValueError` on longer input rather than silently truncating, so this limit is enforced rather than assumed.

Multi-head causal self-attention. A single `nn.Linear` projects x to $3d$ and splits into Q, K, V. Each is reshaped $(B, T, d) \to (B, T, h, d_k)$ and transposed to (B, h, T, d_k) so the head axis sits next

to the batch axis and the per-head matrix products are one batched `matmul`. Scores are

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \qquad \text{output} = AV.$$

The $1/\sqrt{d_k}$ scaling matters because $q \cdot k$ is a sum of $d_k = 64$ products; with unit-variance components its variance grows as d_k, so unscaled scores would reach magnitudes where softmax saturates, producing near-one-hot attention and vanishing gradients from the start. Dividing by $\sqrt{d_k}$ restores unit variance and keeps softmax responsive.

M is an additive mask: 0 on allowed positions, $-\infty$ on future ones, applied *before* softmax so masked positions receive exactly zero weight while each row still sums to one. Heads are concatenated by the inverse transpose-and-reshape and passed through an output projection W_O.

Block. Pre-norm: $x \leftarrow x + \text{attn}(\text{LN}(x))$ then $x \leftarrow x + \text{ffn}(\text{LN}(x))$. Pre-norm keeps an unnormalised residual path from input to output, so gradients reach early layers without passing through ten LayerNorms; at $N = 10$ this is the difference between training stably from step 1 and needing a long warmup to avoid divergence.

Output head. A linear map to 16,000 logits, with its weight matrix *tied* to the input embedding. Tying saves $16000 \times 384 = 6{,}144{,}000$ parameters — untied, the two embedding matrices alone would consume roughly 49% of the 25M budget, leaving too little for the blocks that do the actual modelling. This is precisely why Phase 1 settled on a 16k vocabulary. Training minimises cross-entropy between the logits at position t and the token at $t + 1$, averaged over all positions.

1.4 Empirical verification

Correctness is demonstrated, not asserted. `python -m common.modeling.verify_model` runs six checks; all six pass on the same GPU that trained the models.

Table 2: Verification output. The causality check is the one spec §2.1 requires.

Check	Measured	Result
Causality, positions 0..t	max $\lvert\Delta\rvert$ = 0.000e+00	PASS
Causality, position t+1	max $\lvert\Delta\rvert$ = 7.052e−01	PASS
Attention rows sum to 1	max $\lvert\sum - 1\rvert$ = 1.192e−07	PASS
Strict upper triangle weight	0.000e+00	PASS
Initial loss vs $\ln V$ (toy config, V=256)	5.541 vs 5.545	PASS
Tied embeddings share one tensor	`lm_head.weight is tok_emb.weight`	PASS
`block_size` guard	over-length input raises	PASS
Parameter count	24,086,016 (3.7% under 25M)	PASS

The causality test changes *only* token t+1 and asserts two things: that logits at positions $0 \ldots t$ are bit-for-bit unchanged, and that the logit at t+1 *did* change. Both halves are necessary — a model that ignored its input entirely would pass the first on its own. The initial-loss check is a second, independent probe: an untrained model should assign roughly uniform probability across the vocabulary, so its loss should be $\ln V$. These checks run on a deliberately small configuration (V=256, 2 layers) so they execute in seconds, giving a target of $\ln(256) = 5.545$ against a measured

5.541 — confirming the output layer is neither mis-scaled nor accidentally biased. For the trained models the same reasoning gives a starting point of $\ln(16000) = 9.680$, consistent with the first logged training loss of 9.22 at step 20, by which point the model has already improved on uniform.

2 Pretraining

2.1 Setup

Each model was trained separately on a single NVIDIA Tesla T4 (Kaggle), reading its own pre-tokenised `uint16` array via `np.memmap`.

Table 3: Training configuration, identical for both runs.

Setting	Value
Optimiser	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1
Weight decay applied to	matrices only (not biases or LayerNorm gains)
Peak learning rate	3×10^{-4}
Schedule	linear warmup 500 steps, then cosine decay to 3×10^{-5}
Micro-batch	24 sequences × 512 tokens
Gradient accumulation	8
Effective batch	98,304 tokens/step
Total steps	6,000
Tokens seen	**589,824,000** ($\approx$ 1 epoch)
Gradient clipping	global norm 1.0
Mixed precision	fp16 with `GradScaler`
Evaluation	every 250 steps
Checkpointing	every 500 steps

Two details are worth stating because they were decisions rather than defaults.

Token budget. 6,000 steps was chosen deliberately. The compute-optimal budget for a 24.1M-parameter model is roughly 20 tokens per parameter, or 482M tokens; one epoch of each corpus is 590M, which sits just above optimal while guaranteeing every document is seen exactly once and none twice. Training three epochs would have quadrupled GPU time for a model already past the point where a single epoch of 590M tokens saturates its capacity.

fp16 rather than bf16. The code prefers bf16 — it has fp32's exponent range and needs no loss scaling — but only where the hardware accelerates it, which means Ampere (sm_80) and newer. The T4 is Turing (sm_75) and reports bf16 as *supported* while providing fp16 tensor cores only, so bf16 there runs emulated. Measured on the T4, the emulated bf16 path ran at 11,538 tokens/s against fp16's 37,236 tokens/s — a **3.2**× difference, or 14.2 hours versus 4.4 per model. The selection is therefore made on compute capability, with `GradScaler` enabled for the fp16 path. Losses were unaffected: at step 20 the two paths gave 9.2231 and 9.2226 respectively.

2.2 Checkpoints

Each checkpoint is a `torch.save` dictionary containing model weights, optimiser state, `GradScaler` state, the current step, the best validation loss, the learning rate at that step, and both the model and training configurations — everything §2.2 requires. Two are kept per language: `last.pt` (overwritten on a fixed interval, for resuming) and `best.pt` (written only when validation improves, and the checkpoint all evaluation below uses). Writes are atomic — to a temporary file, then `os.replace` — so an interrupted write cannot leave a truncated checkpoint that a later run mistakes for complete. The hyperparameters are also written beside the checkpoints as plain `config.json`, so they can be read without unpickling a 300 MB file.

2.3 Loss curves

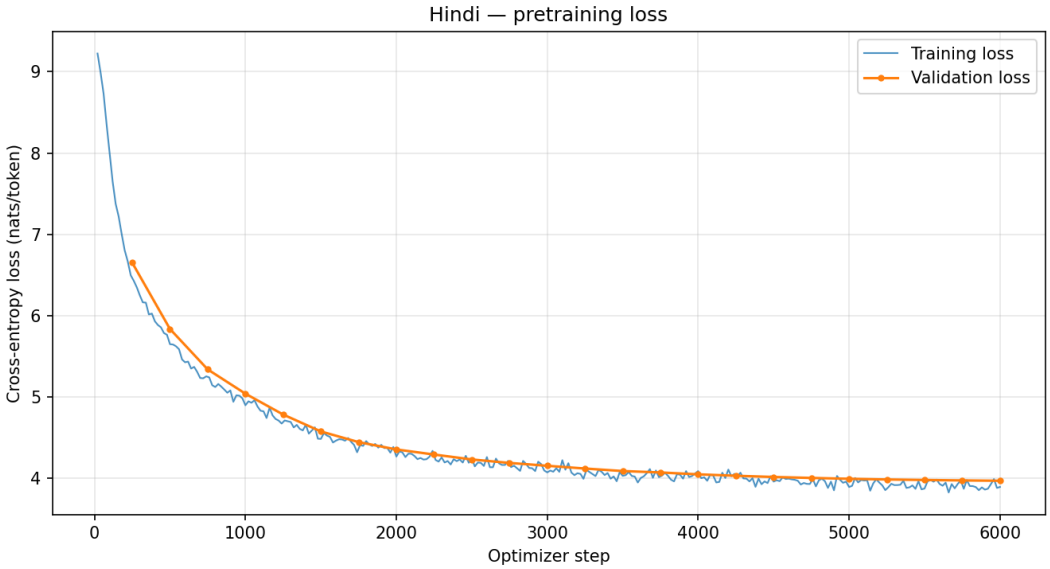


Figure 1: Model H (Hindi) pretraining loss. Training loss logged every 20 steps, validation every 250.

Both curves show the same three phases: a steep drop through warmup as the learning rate ramps to 3×10^{-4}, a long roughly-logarithmic descent, and a flattening as cosine decay reduces the step size. Neither shows a train/validation divergence, which is expected — at one epoch over 590M tokens no example is seen twice, so there is nothing to memorise and dropout at 0.1 is barely needed. Training loss ends at 3.896 (H) and 4.718 (L).

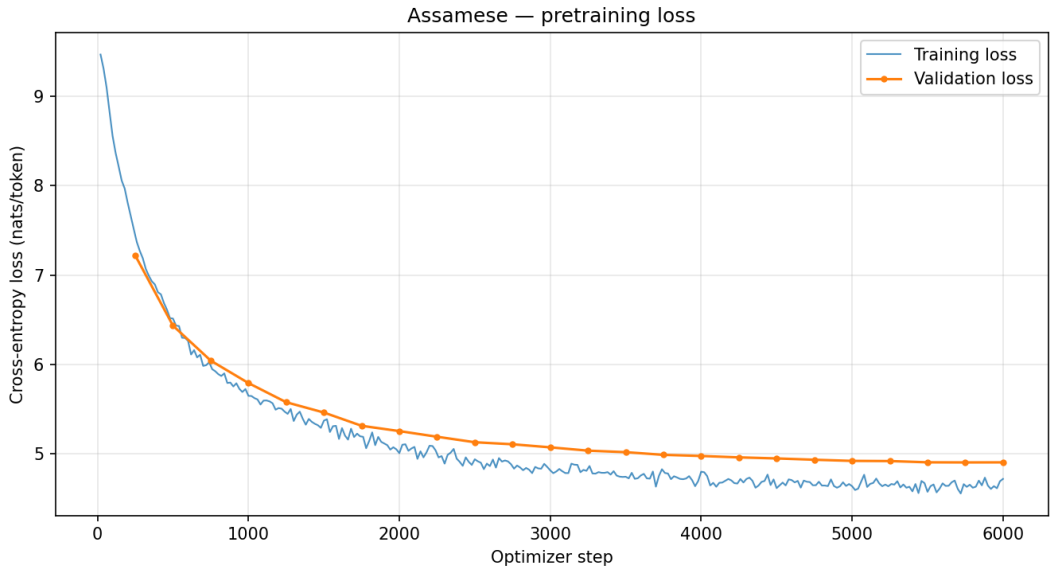


Figure 2: Model L (Assamese) pretraining loss, same axes and schedule.

3 Intrinsic evaluation

All figures below come from `best.pt` scored over the *entire* held-out split using consecutive non-overlapping windows, so every token is counted exactly once. This differs from the in-loop validation number, which is capped at 40 batches for speed.

Both models trained the full 6,000 steps, but `best.pt` is whichever checkpoint minimised validation loss, and for Model L that was step 5,750 (4.9028) rather than step 6,000 (4.9034). The difference is well inside step-to-step noise; it is recorded here because the evaluation JSONs carry `checkpoint_step` and the two models therefore report different values.

Table 4: Intrinsic language-modelling metrics, both models side by side.

	Model H (Hindi)		Model L (Assamese)	
Metric	**val**	**test**	**val**	**test**
Cross-entropy (nats/token)	3.8564	3.8864	4.5478	4.5556
Perplexity	47.30	48.73	94.42	95.16
Bits-per-byte	**0.5537**	**0.5595**	**0.6204**	**0.6238**
Tokens scored	6,991,360	6,811,136	8,361,472	8,169,472
Bytes per token	10.048	10.021	10.576	10.536

Test sits marginally above validation for both models, which is the correct direction: `best.pt` is selected on validation, so the validation figure is slightly optimistic. Had test come out *better* than validation, that would have indicated a leak between splits.

3.1 Why bits-per-byte, and how to read the gap

Perplexity is a per-*token* quantity, and a Hindi token is not the same unit as an Assamese token: the two tokenizers were trained separately and segment text differently (Phase 1 fertility 1.3067 vs 1.6019 tokens per word). Comparing perplexities directly would therefore compare two different questions. Bits-per-byte renormalises to a unit both languages share:

$$\mathrm{BPB} = \frac{\mathrm{CE}_{\text{nats/token}}}{\ln 2 \cdot (\text{bytes per token})}.$$

There is a second, easily-missed subtlety. Perplexity is exp(CE), so it *amplifies* differences, while BPB is linear in CE. The measured cross-entropy gap is 0.6692 nats/token; the same gap reads as **1.95**× in perplexity but **1.11**× in bits-per-byte. Both describe one underlying difference, and quoting the perplexity ratio as though Model L were "twice as bad" would overstate it substantially. The honest statement is that Model L needs about 11% more bits per byte of text than Model H.

Note also that bytes-per-token came out close for the two models (10.02 vs 10.54), so in this particular case BPB and per-token CE happen to rank the models the same way. That is a result, not a given — it means the observed gap is a genuine modelling difference and not a tokenizer artefact.

4 Generation quality

For each model, 64 prompts of 64 tokens were drawn from the held-out test split and continued for 96 tokens under greedy decoding and temperatures 0.5, 1.0 and 1.5. References are the true 96-token continuations that followed those prompts in the corpus.

Table 5: Generation metrics on the test split, both models under every decoding setting. Higher is better except for *rep-4*, where lower is better.

Decoding	Model	BLEU-4	chrF	ROUGE-L	Dist-1	Dist-2	rep-4
Greedy	Model H	1.76	12.16	0.102	0.066	0.128	0.824
	Model L	2.47	11.24	0.059	0.079	0.117	0.856
$T = 0.5$	Model H	2.68	17.41	0.132	0.142	0.353	0.444
	Model L	2.89	14.98	0.081	0.153	0.299	0.563
$T = 1.0$	Model H	1.58	**21.40**	0.116	0.381	0.816	0.072
	Model L	2.97	**22.27**	0.074	0.475	0.881	0.066
$T = 1.5$	Model H	0.12	19.31	0.058	0.588	0.952	0.034
	Model L	0.11	20.73	0.032	0.655	0.983	0.013

4.1 What these metrics do and do not tell us

BLEU-4 is close to uninformative here, and the numbers show why: every value sits between 0.1 and 3.0. BLEU rewards exact 4-gram overlap with a single reference, but open-ended continuation has no single correct answer — a fluent, entirely reasonable continuation that happens to use different words scores zero. It is also brittle for morphologically rich, agglutinative languages:

Assamese inflection means a semantically correct word form can easily miss the reference's exact surface form. BLEU is reported because the spec asks for it, but it should not be read as a quality ranking.

chrF is more informative for exactly the reason BLEU is not. It operates on character n-grams, so it gives partial credit when a model produces a correct stem with a different inflection — common in both languages, and especially in Assamese. The chrF ordering is also more plausible than BLEU's: both models peak at $T = 1.0$ (21.40 and 22.27) rather than at greedy, matching the qualitative observation below that greedy output degenerates.

ROUGE-L measures longest common subsequence, so it tolerates insertions and reordering that BLEU punishes. It shows a consistent H > L gap at every temperature (0.132 vs 0.081 at $T = 0.5$), which aligns with the intrinsic metrics — unlike BLEU, which ranks L above H at three of four settings and contradicts every other measurement. That contradiction is itself evidence for treating BLEU with suspicion here.

4.2 Diversity and repetition

The diversity diagnostics tell a clearer story than any overlap metric. Greedy decoding produces a **4-gram repetition rate of 0.824 (H) and 0.856 (L)** with Distinct-1 near 0.07 — meaning the output is overwhelmingly a small set of tokens cycling. This is the classic degeneration mode of a small model under argmax decoding: once it enters a high-probability loop there is no stochasticity to escape it. Raising temperature monotonically fixes repetition (0.824 → 0.034 for H) and raises diversity (Distinct-1 0.066 → 0.588), at the cost of coherence past $T = 1.0$.

4.3 Qualitative samples

In Model H the greedy repetition loop is visible directly, and the same prompt at $T = 1.0$ yields varied, topically-plausible vocabulary without sustained coherence:

Model H (Hindi) — same prompt, two decoding settings

Prompt विद्या में विशिष्ट विषय 134 अब अनुपयुक्त-पूर्व में मेस्मेरिज़्म और क्लेयरवॉयन्स 135 स्वप्न

Greedy , 187 मनोविज्ञान, 187 मनोविज्ञान, 187 मनोविज्ञान, 187 मनोविज्ञान, ...

$T = 1.0$ साइटैटॉजी, आर्टिफिसोपोटोलॉजी, ेटिकबायोलॉजी, मैकेनिकल, रसायनशास्त्र, वैज्ञानिक अध्ययन और बीमारियों और कमजोर दक्ष

Model L shows the same pattern — a plausible opening that collapses into a loop under greedy decoding, and syntactically well-formed Assamese at $T = 1.0$ with correct script, postpositions and verb morphology, though the content is invented:

Model L (Assamese) — same prompt, two decoding settings

Prompt ৰ মালায়ালাম ৰিমেকত এইবাৰ দেখিবলৈ পোৱা যাব অভিনেতাগৰাকীক। সূত্ৰৰ মতে, ইমৰান অভিনীত এই নতু

Greedy নাম হৈছে "ডাইৰ্ব্বুক" আৰু "ডাইৰ্ব্বুক"। এই ছবিখন মুক্তি পোৱাৰ পিছত, এই ছবিখন মুক্তি পোৱাৰ পিছত, ...

$T = 1.0$ নাম হৈছে পোটাৰ আণ্ডাৰ খান আৰু তেওঁৰ পত্নী ক্ৰমে- উদিত দাস, মুকেশ বিবি ৰহমান। ছবিখনৰ নামটো হৈছে "নাম নিতী সিং"

Assessment. Both models have learned script, morphology and local syntax convincingly: output is always in the correct script, word forms are well-formed, and short phrases read naturally. Neither maintains discourse coherence beyond roughly a clause, and both invent named entities freely. This is the expected competence profile for a 24M-parameter model trained on a single epoch of 590M tokens — local statistics are well captured, long-range semantic planning is not.

5 Attention analysis

Attention was collected from `best.pt` on a real 48-token passage from each model's own held-out split. Two statistics are reported per head:

Normalised entropy. Raw attention entropy is not comparable across positions, because a query at position t can attend over only $t{+}1$ keys and so has a position-dependent maximum of $\ln(t{+}1)$. Dividing by that maximum gives a number in $[0, 1]$ where 1 means perfectly uniform attention over everything visible and 0 means all mass on a single key. Position 0 is excluded, since it can only attend to itself and is trivially degenerate.

Mean attention distance. The average of $|t - j|$ weighted by attention, measuring how far back a query reaches.

Table 6: Layer-mean normalised entropy and mean attention distance. Bold marks each model's local band — the layers where attention contracts to nearby tokens.

	Model H (Hindi)		Model L (Assamese)	
Layer	Entropy	Distance	Entropy	Distance
0	0.94	10.8	0.92	10.9
1	0.90	11.5	0.93	11.5
2	0.72	**4.6**	0.82	8.5
3	0.56	**5.0**	0.72	**4.3**
4	0.60	9.8	0.52	**3.9**
5	0.56	10.8	0.60	9.5
6	0.69	11.7	0.61	11.2
7	0.61	13.1	0.67	11.9
8	0.69	12.5	0.74	12.4
9	0.72	12.1	0.81	11.8

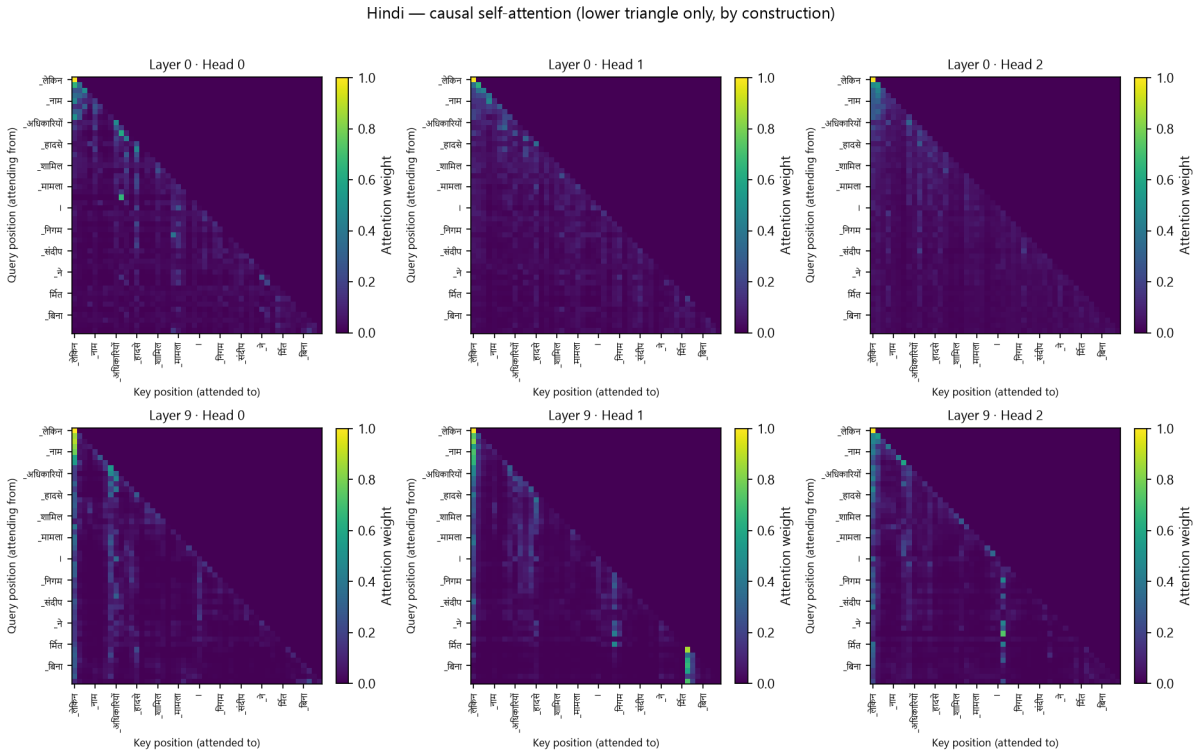


Figure 3: Model H attention, early layer (0) and late layer (9), three heads each. Axis labels are the actual Devanagari tokens of the held-out passage. Everything above the diagonal is exactly zero — the causal mask, visible.

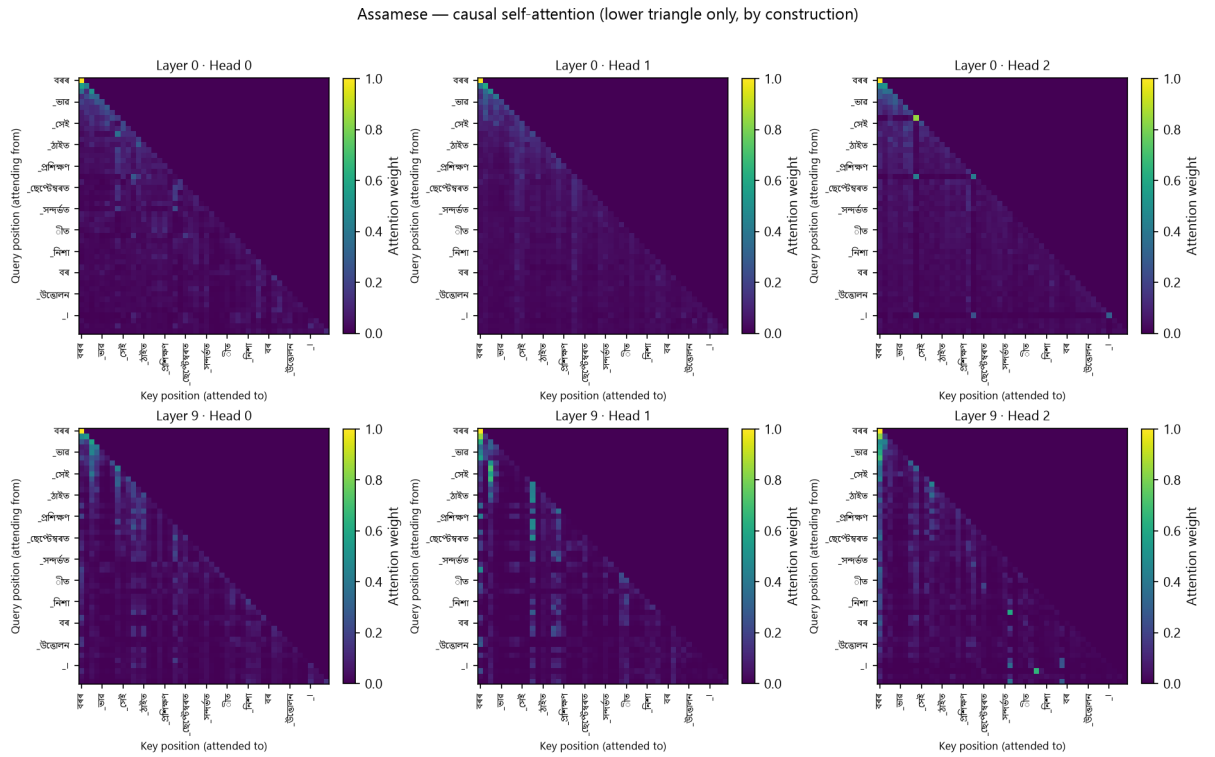


Figure 4: Model L attention on an Assamese passage, same layers and heads.

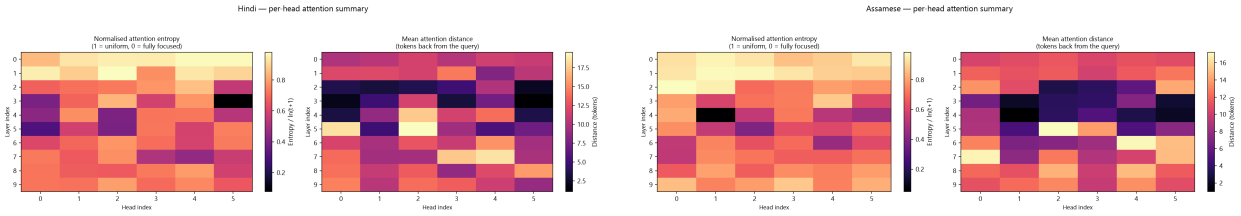


Figure 5: Per-head entropy and mean distance, Model H (left) and Model L (right).

5.1 What the heads learned

The structure is learned, not initial. At step 50 both models showed entropy ≈ 1.0 and distance ≈ 12.0 at every layer — uniform attention over the whole visible context, which is exactly what random initialisation produces. The differentiation reported above is therefore a training outcome.

Both models learn a local band, then widen. Layers 0–1 are broad and near-uniform (entropy 0.90–0.94), a sharp local band follows (H at layers 2–3, L at layers 3–4), and the upper half widens again to distances of 12–13 tokens with intermediate entropy. The interpretation is a pipeline: early layers gather undifferentiated context, middle layers resolve local morphology and word boundaries, later layers integrate across the sequence.

Individual heads specialise far more sharply than layer means suggest. Averaging hides the extremes:

Table 7: Extreme heads, of 60 per model.

Model	Head	Distance	Entropy
H	L3H5 (most local)	1.08	0.077
H	L5H2 (most global)	19.54	0.395
H	L1H2 (most diffuse)	11.65	0.979
L	L4H1 (most local)	0.99	0.049
L	L5H2 (most global)	17.21	0.390
L	L2H0 (most diffuse)	13.59	0.984

Both models independently learned a near-perfect **previous-token head** — H's L3H5 at distance 1.08 and L's L4H1 at 0.99, both with entropy below 0.08, meaning essentially all mass on a single key one position back. Neither was designed; both emerged from the same objective on different data and different scripts, which is a strong indication the mechanism is doing what attention is supposed to do. At the other extreme, both models place their most global head at *the same address*, L5H2 (distance 19.54 and 17.21).

Positional versus content-based. The very low-entropy, low-distance heads are positional: they attend to a fixed offset regardless of what is there. The mid-entropy, high-distance heads in layers 5–9 are content-based — in the heatmaps they show vertical stripes, where many query positions all attend to one particular earlier token, which is attention keyed on *what* that token is rather than where.

5.2 Where the two models differ

Three differences stand out. First, **Model L's local band sits one layer deeper** (3–4 versus 2–3) and is more pronounced (distance 3.9 versus 4.6). Second, **Model L's heads are sharper overall** — minimum layer-mean entropy 0.52 against H's 0.56, and its sharpest head reaches 0.049 against 0.077. Third, **Model L's final layers stay more diffuse** (entropy 0.81 at layer 9 versus 0.72).

A plausible reading, offered as interpretation rather than established fact: Assamese's higher fertility (1.6019 versus 1.3067 tokens per word) means a word is split into more subword pieces, so more of the model's capacity must go to reassembling words from fragments before any higher-level structure can be built. That would explain both the deeper, sharper local band — more work to do at the sub-word level — and the more diffuse top layers, if less capacity remains for confident long-range integration. Testing this properly would require an ablation over vocabulary size, which is outside Phase 2's scope.

6 Resource-level comparison

Table 8: Model H versus Model L across all resource and outcome dimensions.

Dimension	Model H (Hindi)	Model L (Assamese)
Documents after dedup	1,010,907	846,348
Words after dedup	464,048,535	384,237,720
Word retention through dedup	97.3%	87.9%
Tokenizer fertility (tokens/word)	1.3067	1.6019
Characters per token	3.9288	4.0194
Train-split tokens	589,952,668	591,255,819
Total corpus tokens (train + val + test)	603,733,652	607,764,930
Training tokens seen	589,824,000	589,824,000
Parameters	24,086,016	24,086,016
Training time	290 min	263 min
Test perplexity	48.73	95.16
Test bits-per-byte	**0.5595**	**0.6238**

The comparison is unusually clean, because **the confound that normally dominates higher- versus lower-resource comparisons is absent**. Both models saw the same 589,824,000 training tokens, have identical architectures and parameter counts, and trained under identical hyperparameters. The Phase 1 collection effort deliberately expanded the Assamese corpus until it matched Hindi's token count — the two totals differ by 0.2%. So the 11% BPB gap cannot be attributed to data *quantity*. Three explanations remain.

Tokenizer fertility. Assamese needs 1.6019 tokens per word against Hindi's 1.3067, i.e. 22.6% more. At an equal token budget, Model L therefore saw roughly 22.6% *fewer words* — 384M against 464M after dedup. Equal tokens is not equal linguistic content. This is the single largest identified factor, and it is a direct consequence of Assamese being lower-resource: the 16k vocabulary was trained on a smaller and less varied text pool, so it captures fewer whole words as single units.

Corpus composition. Word retention through deduplication was 97.3% for Hindi but 87.9% for Assamese — Assamese had substantially more duplicated material to remove, which indicates a narrower pool of genuinely distinct sources. The Assamese corpus also leans more heavily on a handful of news domains and aggregated web sources, where Hindi draws across Wikipedia, Wikisource, books and news more evenly. Less topical variety per token means a harder generalisation problem on held-out text.

Script and morphology. Assamese is morphologically richer, with more inflectional variation per lemma. Combined with subword fragmentation, the model must model more distinct surface forms from the same effective content.

What this implies for the "low-resource" framing. The interesting result is how *small* the gap is once measured properly. A naive reading of perplexity (48.73 versus 95.16) suggests Model L is roughly half as good. The byte-normalised comparison puts the gap at 11%. Most of the apparent difference is an artefact of comparing per-token quantities across non-equivalent tokenizers and of perplexity's exponential scale. The genuine disadvantage of the lower-resource

language, at matched token budget and matched architecture, is real but modest — and the largest single contributor to it is the tokenizer, which is itself downstream of data availability.

7 Reproduction and artifacts

7.1 In this repository (branch phase-2)

Shared code lives in `common/`, grouped by pipeline stage: `sources/` (one module per data source), `clean/` (normalisation, language ID, quality filtering), `corpus/` (dedup, splits, decontamination), `tokenization/` (SentencePiece training and pre-tokenisation) and `modeling/` (everything in this phase). Only `io.py` and `pipeline.py` sit at the top level. Per-language artifacts stay inside `hindi/` and `assamese/`, which share no data, tokenizer, vocabulary or weights.

Path	Contents
`common/modeling/model.py`	Transformer, attention, blocks
`common/modeling/train.py`	training loop, schedule, checkpointing
`common/modeling/data.py`	memory-mapped batching over the token arrays
`common/modeling/evaluate.py`	CE / PPL / BPB, BLEU, chrF, ROUGE-L, diversity
`common/modeling/attention.py`	attention collection, statistics, figures
`common/modeling/verify_model.py`	the six correctness checks of §1.4
`common/tokenization/tokenize_corpus.py`	corpus → `uint16` arrays
`<lang>/configs/model.json`	architecture, per language
`<lang>/model/`	training log, loss curve, config, parameter report
`<lang>/eval/`	metrics JSON (val + test), attention figures and stats
`kaggle/`	the notebook that ran pretraining, and the payload stager

7.2 On Google Drive

Checkpoints are too large for git (299.7 MB each). The folder contains `hindi/` and `assamese/`, each with `best.pt` and `last.pt`, plus a `README.txt` documenting the payload and a tested three-line loading snippet:

https://drive.google.com/drive/folders/1bIOG67ZM5iARPTvk3FGR51DxqIIxlLLn?usp=sharing

7.3 Reproducing

1. `python -m common.tokenization.tokenize_corpus --lang hindi` (and `--lang assamese`).
2. `python -m common.modeling.verify_model` — the six checks must pass.
3. Run `kaggle/phase2_train_kaggle.ipynb` once per language on a GPU.
4. `python -m common.modeling.evaluate --lang <lang> --split test`.
5. `python -m common.modeling.attention --lang <lang>`.

Steps 4 and 5 are also run by the notebook. Attention figures were regenerated locally rather than taken from the notebook output, because the Kaggle image ships no Indic font and falls back to positional tick labels; the statistics are bit-identical either way, only the axis labels differ.